FINAL YEAR ENGINEERING PROJECT

Smart Medicine Availability & Intelligent Janaushadhi Recommendation System

PROJECT FOLDER STRUCTURE

Project	Smart Medicine Availability & Intelligent Janaushadhi Recommendation System
Document	PROJECT FOLDER STRUCTURE
Version	v1.0.0
Date	02 July 2026

Every folder explained — Purpose, Responsibility, and When to Add Files

Root Level

```
Frontend/
■■■ docs/            ← Developer documentation (this folder)
■■■ dist/            ← Production build output (generated by npm run build)
■■■ node_modules/    ← Installed packages (never edit manually)
■■■ public/          ← Static files served as-is
■■■ src/             ← All source code lives here
■■■ .env             ← Environment variables (local, not committed)
■■■ .env.example     ← Template for .env — safe to commit
■■■ .gitignore       ← Files Git should ignore
■■■ eslint.config.js ← Linting rules
■■■ index.html       ← HTML shell that React mounts into
■■■ package.json     ← Project metadata and dependencies
■■■ package-lock.json ← Exact dependency versions (auto-generated)
■■■ vite.config.js   ← Vite bundler configuration
```

`src/` — All Source Code

Every file a developer writes goes here.

`src/components/`

Purpose: Reusable UI building blocks shared across multiple pages.

Rule: A component in this folder should NOT know about pages, routes, or business logic. It only knows about its props and how to render them.

When to add files here: When you create a UI element that is (or could be) used in more than one place.

`src/components/ui/`

Purpose: The smallest UI atoms — the basic building blocks of the design system.

Files:

File	Purpose
Button.jsx	Primary, secondary, danger, outline, ghost variants
Badge.jsx	Small labels (In Stock, Generic, Admin)
Avatar.jsx	User profile picture with fallback initials
Divider.jsx	Horizontal or vertical separator line
IconButton.jsx	Square button containing only an icon
index.js	Barrel export: export { Button, Badge, ... }

Why it exists: Without this folder, every page would create its own button. You'd have 20 different buttons with 20 different styles. This folder gives one consistent button used everywhere.

`src/components/forms/`

Purpose: Form input controls used in every form across the app.

Files:

File	Purpose
Input.jsx	Text input with label, error, and icon support
PasswordInput.jsx	Password input with show/hide toggle
PasswordStrength.jsx	Password strength indicator bar
Textarea.jsx	Multi-line text input
Select.jsx	Dropdown select with custom styling
Checkbox.jsx	Styled checkbox with label
RadioGroup.jsx	Group of radio buttons
Toggle.jsx	On/off switch
OtpInput.jsx	6-box OTP entry with auto-advance
FormField.jsx	Wrapper that adds label + error message to any input
index.js	Barrel export

Why it exists: Consistent form styling and behaviour across Login, Register, Inventory, Profile, etc.

`src/components/cards/`

Purpose: Card components that display data in a visual card layout.

Files:

File	Purpose
MedicineCard.jsx	Basic medicine display card
SearchResultCard.jsx	Rich card for search results (badges, compare, save, share)
PharmacyCard.jsx	Pharmacy information card
NotificationCard.jsx	Single notification item card
InfoCard.jsx	Generic info/stat card for dashboards
index.js	Barrel export

Why it exists: Cards are used in search results, dashboards, pharmacies list, and notifications. One consistent card design avoids visual inconsistency.

`src/components/common/`

Purpose: General-purpose utility components used across the entire app.

Files:

File	Purpose
Breadcrumb.jsx	Navigation trail (Home > Medicines > Paracetamol)
SearchBar.jsx	Search input with icon and submit
Pagination.jsx	Page navigation with prev/next/page numbers
AppErrorBoundary.jsx	Catches React crashes and shows error page
index.js	Barrel export

Why it exists: These components don't fit neatly into ui/forms/cards — they are standalone utility components needed everywhere.

`src/components/feedback/`

Purpose: Components that communicate loading, empty, and error states to users.

Files:

File	Purpose
Spinner.jsx	Animated loading circle
Skeleton.jsx	Grey animated placeholder while loading
EmptyState.jsx	"No results found" screen with icon and message
ErrorState.jsx	"Something went wrong" screen with retry button
index.js	Barrel export

Why it exists: Every page that loads data needs loading, empty, and error states. These components make it easy without duplicating the UI logic.

`src/components/dialogs/`

Purpose: Overlay components that appear on top of the page.

Files:

File	Purpose
Modal.jsx	Configurable modal dialog (header, body, footer)
ConfirmDialog.jsx	Yes/No confirmation dialog (e.g. "Are you sure you want to delete?")
index.js	Barrel export

`src/components/navigation/`

Purpose: Navigation chrome — the parts of the UI that help users move around.

Files:

File	Purpose
Navbar.jsx	Top navigation bar for public pages
Footer.jsx	Site footer with links

File	Purpose
Sidebar.jsx	Left sidebar for authenticated dashboards
TopBar.jsx	Top header bar inside authenticated layouts
index.js	Barrel export

`src/components/layout/`

Purpose: Layout utilities that control the structure of page content.

Files:

File	Purpose
Container.jsx	Max-width wrapper with responsive horizontal padding
PageHeader.jsx	Page title + breadcrumb + optional action button
SectionHeader.jsx	Section heading + subtitle within a page
index.js	Barrel export

`src/pages/`

Purpose: The actual screens of the application. Each page maps to a URL route.

Rule: Pages are NOT reusable. They are assembled from reusable components. Business logic lives in hooks and services, not pages.

When to add files here: When you are adding a new URL route / screen.

```
pages/
■■■ home/              → /               (public landing)
■■■ auth/              → /login, /register, /verify-otp, etc.
■■■ dashboard/         → /dashboard      (patient)
■■■ search/            → /search         (medicine search)
■■■ results/           → /search/results
■■■ medicine/          → /medicine/:id
■■■ generic/           → /medicine/:id/generic
■■■ pharmacies/        → /pharmacies/nearby
■■■ notifications/     → /notifications
■■■ profile/           → /profile
■■■ pharmacy/          → /pharmacy/dashboard, /inventory
■■■ admin/             → /admin/dashboard, /users, etc.
■■■ errors/            → offline, server-error, session-expired
■■■ NotFoundPage.jsx   → 404 catch-all
■■■ UnauthorizedPage.jsx → 403 access denied
```

sections/ subfolders: Large pages (Home, MedicineDetails, Generic, NearbyPharmacies) split their content into section components inside a `sections/` subfolder. This keeps the main page file clean.

`src/layouts/`

Purpose: Persistent wrappers around groups of pages. The layout renders the nav/sidebar/footer, and the page renders inside using React Router's `<Outlet />`.

Files:

File	Route Group
MainLayout.jsx	Public pages (Home)
AuthLayout.jsx	Auth pages (Login, Register, OTP)
UserLayout.jsx	Patient/Doctor pages
PharmacyLayout.jsx	Pharmacist pages
AdminLayout.jsx	Admin pages

`src/routes/`

Purpose: React Router configuration and route guards.

Files:

File	Purpose
AppRouter.jsx	Defines the entire route tree
ProtectedRoute.jsx	Blocks access if not authenticated or wrong role
PublicRoute.jsx	Redirects authenticated users away from auth pages

---

`src/contexts/`

Purpose: React Context — global state that any component in the tree can read.

When to add files here: When you have state that is needed by many components at different levels and you want to avoid passing it as props through many layers (prop-drilling).

Files:

File	Global State
AuthContext.jsx	currentUser, isAuthenticated, login, logout
ThemeContext.jsx	theme, isDark, toggleTheme
UserContext.jsx	userProfile, updateProfile

`src/hooks/`

Purpose: Custom React hooks — reusable stateful logic extracted from components.

When to add files here: When you find yourself writing the same `useState`/`useEffect` logic in multiple components.

Files:

File	Purpose
useDebounce.js	Delay a value (search input optimisation)
useLocalStorage.js	Sync state with localStorage

File	Purpose
useOnlineStatus.js	Track browser online/offline
useSessionGuard.js	Auto-logout after inactivity

`src/services/`

Purpose: All API calls. Every HTTP request in the app goes through a service.

When to add files here: When you add a new group of API endpoints (e.g., `reportService.js`).

Files:

File	API Endpoints
authService.js	/auth/* — login, register, OTP
userService.js	/users/* — profile, admin users
medicineService.js	/medicines/* — search, details, alternatives
pharmacyService.js	/pharmacies/* — nearby, inventory
inventoryService.js	/inventory/* — stock management

`src/config/`

Purpose: Application configuration — setup files, not business logic.

Files:

File	Purpose
axiosClient.js	Axios instance with JWT interceptors and 401 handling
env.js	Environment variable wrapper
queryClient.js	TanStack React Query client with default caching options

---

`src/constants/`

Purpose: Magic strings and magic numbers replaced with named constants.

When to add files here: Any time you would write a hardcoded string (a route path, a role name, a storage key) in more than one place.

Files:

File	Contains
app.js	USER_ROLES, STORAGE_KEYS, TOAST, PAGINATION, HTTP_STATUS
routes.js	All route paths: ROUTES.USER.DASHBOARD, ROUTES.ADMIN.USERS, etc.
navConfig.js	Sidebar navigation items for each role

`src/utils/`

Purpose: Pure utility functions — functions that take input and return output with no side effects.

When to add files here: When you have logic that does not need React, has no state, and can be tested independently.

Files:

File	Functions
formatters.js	formatCurrency, formatDate, toTitleCase, truncate
validators.js	isValidEmail, isValidIndianPhone, isStrongPassword, isValidOtp
authSchemas.js	Zod schemas: loginSchema, registerSchema, otpSchema, etc.
storage.js	storage.get, storage.set, storage.remove, storage.clear

---

`src/styles/`

Purpose: Design system configuration in JavaScript.

Files:

File	Purpose
index.css	Tailwind v4 @theme {} CSS custom properties — the master design tokens
tokens.js	JS mirror of design tokens for programmatic use (charts, inline styles)
theme.js	Light/dark theme config objects
icons.js	Common icon re-exports